

Informe Técnico

Household Power MLOps

*Pronóstico del consumo eléctrico residencial mediante un pipeline
MLOps end-to-end (ingesta, validación, modelado, despliegue y monitoreo)*

Grupo 7

Breidy Bonilla (@BreidyBR)

Oscar Eduardo Sánchez Barahona (@oscar2903)

Procomer — Módulo 6: MLOps

Septiembre 2026

Repositorio: github.com/BreidyBR/Household-Power-Mlops

1. Introducción y objetivo

Este informe documenta el desarrollo completo del proyecto Household Power MLOps, cuyo propósito es diseñar, implementar y documentar una solución de Machine Learning bajo principios de MLOps, cubriendo todo el ciclo de vida: desde la ingesta de datos crudos hasta el despliegue, monitoreo y la propuesta de una estrategia de reentrenamiento.

Más que un modelo aislado, el objetivo del proyecto fue construir un sistema reproducible, versionado, automatizable, desplegable, observable, mantenible y auditable, es decir, un pipeline que cualquier persona del equipo (o un tercero) pueda clonar y ejecutar de principio a fin obteniendo los mismos resultados.

1.1 Problema de Machine Learning

El proyecto corresponde a un problema de series de tiempo: pronosticar el consumo eléctrico residencial futuro a partir de su historial. La frecuencia de trabajo (horaria) y el horizonte de predicción (24 horas) no se asumieron de antemano, se determinaron y justificaron a partir de la evidencia encontrada durante el Análisis Exploratorio de Datos (EDA), como se detalla en la sección 4.

1.2 Dataset

Se utilizó el dataset público Individual Household Electric Power Consumption, del UCI Machine Learning Repository, con 2,075,259 registros de mediciones de consumo eléctrico domiciliario tomadas cada minuto entre diciembre de 2006 y noviembre de 2010.

Decisión de diseño

No versionar los datos crudos en Git: el archivo original supera el límite práctico de un repositorio Git; en su lugar, se implementó un proceso de ingesta reproducible (sección 3) que descarga y valida el dataset localmente en `data/raw/`, excluido vía `.gitignore`.

2. Arquitectura MLOps

El proyecto sigue un flujo end-to-end con las siguientes etapas, cada una con criterios de éxito/fallo explícitos antes de avanzar a la siguiente:

- Fuente de datos (UCI) → Data Ingestion
- Data Ingestion → Raw / Bronze (`data/raw/`)
- Raw → Data Validation (8 Data Quality Gates) → si FAIL: se bloquea el pipeline; si PASS/WARNING: continúa
- Data Cleaning (tipado, interpolación acotada)
- Feature Pipeline (calendario, lags, rolling, target a 24h)
- Training (comparación de modelos) → Evaluation (TimeSeriesSplit)
- MLflow Tracking + Model Registry (Experiment → Candidate → Validation → Production)
- Dockerize → Model API (FastAPI)

- Producción → Monitoring (Data Drift, Model Performance, System Metrics, Data Quality)
- Retraining Trigger (decisión basada en drift + degradación de desempeño)

Decisión de diseño

Separar claramente cada etapa en su propio módulo de src/: facilita auditar, probar y reproducir cada paso de forma independiente (por ejemplo, correr solo la validación sin tener que re-entrenar el modelo), y permite que ambos integrantes trabajaran en paralelo sobre distintas etapas sin pisarse el código.

3. Data Ingestion

Qué se hizo

Implementado en src/ingestion/ingest.py. El proceso automatiza:

- Descarga del dataset desde el repositorio oficial de UCI Machine Learning.
- Extracción del archivo comprimido original.
- Almacenamiento en data/raw/.
- Validación básica de existencia y tamaño del archivo.
- Conteo de registros y cálculo de hash SHA-256.
- Generación de metadata de la ingesta (fuente, fecha UTC, cantidad de registros, tamaño, hash).

Decisiones tomadas y justificación

- Idempotencia: si el dataset ya existe en data/raw/, el proceso evita volver a descargarlo. Ahorra tiempo y ancho de banda en ejecuciones repetidas del pipeline.
- Hash SHA-256 como huella de versión de datos: permite verificar en cualquier momento que el archivo local no fue corrompido ni modificado, y sirve de base para el data_version que luego se registra en cada run de MLflow (trazabilidad del dato y del modelo).

4. Data Validation

Qué se hizo

Implementado en src/validation/validate.py: ocho Data Quality Gates automáticas sobre el dataset crudo, cada una con veredicto PASS, WARNING o FAIL:

- Columnas obligatorias presentes.
- Número mínimo de filas.
- Timestamps válidos.
- Duplicados bajo umbral.
- Missing values bajo umbral.
- Global_active_power ≥ 0 (regla de plausibilidad física).
- Frecuencia temporal correcta (1 minuto).
- Gaps temporales controlados.

Cada corrida genera `reports/data_quality/data_quality_report.json` como evidencia auditable, con el veredicto global y el detalle de las ocho reglas.

Decisiones tomadas y justificación

- Umbrales calibrados con el dataset real (no arbitrarios): cada límite se justificó en `notebooks/01_data_quality_cleaning.ipynb` observando el comportamiento real de los 2,075,259 registros, en vez de copiar reglas genéricas.
- Distinción FAIL vs. WARNING: un FAIL bloquea el pipeline (indica un problema que invalidaría el resto del proceso); un WARNING se registra pero no detiene la ejecución, porque corresponde a una desviación menor que no compromete la validez del dataset.
- Modo `--strict` opcional: pensado para integrarlo a un futuro pipeline de CI/CD, donde un FAIL debe traducirse en un código de salida distinto de cero.

5. Data Cleaning

Qué se hizo

Implementado en `src/cleaning/clean.py`, convirtiendo en un script reproducible las decisiones investigadas en `notebooks/01_data_quality_cleaning.ipynb` (Parte II):

- Construcción del índice temporal a partir de las columnas Date y Time.
- Conversión de las siete variables eléctricas a tipo numérico, tratando el símbolo '?' como valor faltante.
- Análisis de la distribución temporal de los missing (rachas cortas vs. largas).
- Interpolación temporal lineal únicamente sobre rachas de valores faltantes ≤ 60 minutos.
- Validación de que la limpieza no elimina filas ni introduce inconsistencias (índice creciente, sin duplicados, sin negativos).

El resultado se guarda en `data/interim/household_power_cleaned.parquet`, preservando las 2,075,259 filas originales.

Decisiones tomadas y justificación

- Interpolar solo rachas cortas (≤ 60 min): interpolar linealmente rachas largas fabricaría una tendencia artificial inexistente en la realidad.
- Dejar como NaN las 9 rachas largas (~25,691 min, hasta ~5 días): su tratamiento definitivo se difirió intencionalmente a Feature Engineering, donde se decide en conjunto con la frecuencia de resampling — evita tomar una decisión de imputación antes de saber a qué granularidad temporal se va a trabajar.

6. EDA temporal

Qué se hizo

Documentado en notebooks/02_eda.ipynb, sobre el dataset ya limpio. Se estudiaron:

- Estadísticos descriptivos y distribución de las variables eléctricas.
- Evolución temporal de Global_active_power.
- Patrones de consumo por hora del día, día de la semana y mes.
- Tendencia mediante medias móviles y estacionalidad mediante descomposición de la serie.
- Autocorrelación horaria.
- Valores extremos y su distribución temporal.
- Gaps temporales y duración de los períodos sin información.
- Relaciones entre las variables eléctricas (correlación).

Hallazgos clave

- Consumo con mayor actividad en la mañana y especialmente en la tarde-noche; niveles promedio más altos el fin de semana; patrón anual con mayor consumo a inicio y final de año.
- Autocorrelación fuerte con ciclos de aproximadamente 24 horas y sus múltiplos, evidencia que justificó directamente el uso de lags horarios y el horizonte de 24h en Feature Engineering.
- 94,925 observaciones extremas en Global_active_power (4.63 % del total, vía IQR), concentradas en períodos habituales de mayor consumo, por lo que no se eliminaron automáticamente.
- 421 horas completamente sin información tras resampling horario, agrupadas en 8 gaps (máximo 119 horas).
- Global_intensity correlaciona casi perfectamente con Global_active_power (0.999) — posible redundancia a evaluar en la selección de features.

Decisiones tomadas y justificación

- No eliminar automáticamente los valores extremos: su concentración en horarios de alto consumo sugiere que gran parte corresponde a comportamiento real, no a errores de medición; eliminarlos habría sesgado el modelo hacia consumos promedio e ignorado picos reales que sí interesa predecir.
- No imputar de forma general los gaps largos durante el EDA: esa decisión se pospuso deliberadamente a Feature Engineering, donde se define junto con la estrategia de imputación causal.

7. Feature Engineering

Qué se hizo

Investigado y justificado en notebooks/03_feature_engineering.ipynb, trasladado a pipeline reproducible en src/features/build_features.py, partiendo del dataset limpio:

- Resampling a frecuencia horaria.

- Imputación estacional causal en cascada (de 168h a 24h), usando sólo información pasada.
- Variable objetivo: `Global_active_power` con horizonte de 24 horas.
- Features de calendario: `hour`, `day_of_week`, `month`, `is_weekend`, `hour_sin`, `hour_cos`.
- Lags: 1, 2, 3, 24, 48 y 168 horas.
- Rolling mean/std en ventanas de 3, 6, 24 y 168 horas, calculadas sobre la serie desplazada (`shift(1)`).
- Verificación explícita de ausencia de leakage: cada lag corresponde exactamente a su desplazamiento, el target mira 24h hacia adelante, y ninguna feature coincide sospechosamente con el valor contemporáneo.

Resultado: `data/processed/features_hourly.parquet`. Queda un dataset con 33,976 filas × 20 features + target (21 columnas). Se eliminan 613 filas: 168 iniciales sin historia suficiente para `lag_168h`, 24 finales sin horizonte a `t+24h`, y 421 cuyo target correspondía a una observación originalmente faltante.

Decisiones tomadas y justificación

- Frecuencia horaria: conserva el patrón intradía identificado en el EDA y coincide con las unidades ya definidas para lags y rolling windows.
- Imputación en cascada de 168h a 24h, solo con información pasada: evita cualquier fuga de información futura hacia las features; en este dataset las 421 horas faltantes se resolvieron todas con el valor de 168h antes, sin necesitar el respaldo de 24h.
- Target construido desde la serie observada, NO desde la serie imputada (corrección aplicada durante el desarrollo): la primera versión del pipeline construía el target también a partir de valores imputados, generando 421 targets sintéticos (fabricados, no medidos realmente). Se corrigió separando explícitamente la serie observada (para el target) de la serie imputada (solo para features), evitando que el modelo aprendiera a predecir un valor que en realidad nunca fue medido.
- Lags y ventanas elegidos por evidencia, no por defecto: cada lag (1, 2, 3, 24, 48, 168h) corresponde a un pico real de autocorrelación observado en el EDA, no a una elección arbitraria.

8. Training y Evaluation

Qué se hizo

Documentado en `notebooks/04_training.ipynb`. Los datos se dividen respetando estrictamente el orden temporal (sin shuffle) en train/validación/test. Como referencia se usa un baseline estacional (consumo observado 24h antes). Se evaluaron:

- Regresión Lineal
- Random Forest

- Gradient Boosting
- HistGradientBoosting
- Random Forest regularizado (versión adicional, ver decisión abajo)

Resultados — validación temporal (TimeSeriesSplit, 5 particiones)

Modelo	MAE promedio (kW)	RMSE promedio (kW)
Random Forest regularizado (mejor candidato)	0.5227	0.7106

Resultados — evaluación final sobre test (modelo reentrenado con train+validation)

Métrica	Valor
MAE	0.43 kW
RMSE	0.59 kW
sMAPE	46.53 %

El conjunto de test permaneció completamente separado durante todo el proceso de selección de modelo, usándose únicamente para la evaluación final. En comparación con el baseline estacional, el modelo final reduce considerablemente el error. Random Forest representa bien el comportamiento general del consumo, aunque presenta mayor dificultad para reproducir picos elevados y cambios bruscos.

Decisiones tomadas y justificación

- Split estrictamente temporal (sin shuffle): mezclar aleatoriamente observaciones de una serie de tiempo permitiría que el modelo vea el futuro durante el entrenamiento. Se validó con TimeSeriesSplit precisamente para comprobar el comportamiento del modelo en distintos períodos sin alterar el orden cronológico.
- Agregar una versión regularizada de Random Forest: la configuración inicial mostró sobreajuste; se limitó la profundidad de los árboles y el número mínimo de observaciones por división/hoja, lo que mejoró la generalización.
- Criterio de selección explícito: menor MAE promedio en TimeSeriesSplit, confirmado en test: Random Forest regularizado fue consistentemente el mejor candidato tanto en la comparación de validación como en la validación temporal, no se eligió por intuición sino por comparación reproducible.
- Reentrenar con train+validation antes de la evaluación final: una vez seleccionado el modelo, se aprovecha toda la información disponible antes del test (que se mantuvo intacto), maximizando los datos de entrenamiento sin comprometer la validez de la evaluación final.

9. MLflow Tracking y Model Registry

Qué se hizo

Implementado en `src/training/experiment.py` y `src/training/train.py`. `experiment.py` no vuelve a decidir qué modelo usar, instrumenta con MLflow la misma comparación ya investigada en el notebook de training, registrando cada modelo como un run independiente con:

- **Parameters:** `algorithm`, hiperparámetros, `feature_set`, `random_seed`, `data_version` (hash SHA-256 del dataset).
- **Metrics:** MAE, RMSE, sMAPE.
- **Artifacts:** modelo entrenado, gráfico de residuales y configuración completa en JSON.

`train.py` loggea el run final (modelo reentrenado con train+validation, evaluado en test) y lo registra en el Model Registry, siguiendo el ciclo:

Experiment (6 runs) → Candidate (run final de `train.py`) → Validation
(TimeSeriesSplit, ya superada) → Production (promoción explícita)

Decisiones tomadas y justificación

- Tracking store en SQLite en vez de file-store: a partir de MLflow ≥ 3.0 el backend de tracking de solo archivos quedó en modo mantenimiento; se migró a `sqlite:///mlflow.db`, manteniendo `mlruns/` para los artefactos (modelos, gráficos).
- `random_seed` como parámetro logueado explícitamente: requisito explícito de la sección de tracking del enunciado: cada run debe registrar como mínimo `algorithm`, `hyperparameters`, `feature_set`, `random_seed` y `data_version`, para que cualquier resultado sea auditable y reproducible.
- `archive_existing_versions=True` al promover un modelo a Production (bug encontrado y corregido): sin este parámetro, MLflow no retira automáticamente el estado "Production" de versiones anteriores: al reentrenar, se detectó que la versión anterior seguía marcada como Production simultáneamente con la nueva. Se corrigió y se verificó reentrenando tres veces seguidas, confirmando que solo la versión más reciente queda en Production y las anteriores se archivan automáticamente.
- `mlflow.db` y `mlruns/` no se versionan en Git: son artefactos regenerables (igual que `data/raw/` o `models/` antes de la excepción aplicada); cada integrante los genera localmente corriendo los scripts del pipeline.

10. API de Inferencia y Testing

Qué se hizo

Implementada en `api/main.py`. Sirve el modelo registrado en MLflow como `household-power-forecaster`, y expone:

- **GET /health:** estado del servicio y si el modelo cargó correctamente.
- **GET /model-info:** nombre, versión y stage del modelo servido en ese momento.
- **POST /predict:** recibe las 20 features del modelo y devuelve forecast, horizon y model_version.
- **GET /metrics:** métricas operativas acumuladas del servicio (ver sección 12).

Testing

16 pruebas automatizadas en `tests/`, distribuidas en tres archivos:

- **test_data.py:** esquema, tipos, rangos y ausencia de missing sobre el dataset supervisado final.
- **test_model.py:** carga el modelo real desde el Model Registry y verifica que un input válido produzca un pronóstico numérico físicamente plausible, y que un input inválido genere un error explícito, no un resultado silencioso.
- **test_api.py:** usa TestClient de FastAPI para confirmar que una petición válida responde 200 con el schema esperado, y que peticiones inválidas (campo faltante, tipo incorrecto, cuerpo vacío) responden 422.

Decisiones tomadas y justificación

- Carga del modelo dependiente del entorno de ejecución: en desarrollo local, la API consulta el modelo marcado como Production en el Model Registry de MLflow; dentro de Docker, carga directamente el artefacto `models/random_forest_model.joblib` vía la variable `MODEL_PATH`. Esto permite que el contenedor funcione de forma completamente autónoma, sin depender de un servidor MLflow corriendo en paralelo.
- Validación automática con Pydantic: un input con tipo incorrecto o campo faltante es rechazado por FastAPI/Pydantic antes de llegar al modelo, devolviendo 422 (Unprocessable Entity) en vez de un error 500 o, peor, una predicción silenciosa sobre datos corruptos.
- `test_model.py` exige que un input inválido falle explícitamente: un modelo que “predice cualquier cosa” ante datos corruptos es más peligroso que uno que falla de forma visible, se verifica activamente ese comportamiento, no se asume.

11. Docker

Qué se hizo

La API y el modelo final se contenerizan mediante Dockerfile, usando `python:3.14-slim` como base y `requirements-api.txt` (dependencias mínimas de producción). Se incorpora el artefacto `models/random_forest_model.joblib` generado por `train.py`, y `MODEL_PATH` le indica a la API dónde cargarlo dentro del contenedor.

```
docker build -t grupo7-mlops .  
docker run --rm -p 8000:8000 --name grupo7-mlops-api grupo7-mlops
```

La ejecución se validó llamando a `/health`, `/model-info` y `/predict` dentro del contenedor, confirmando un pronóstico correcto usando el modelo empaquetado en la imagen.

Decisiones tomadas y justificación

- `requirements-api.txt` separado del `requirements.txt` general: evita instalar en producción dependencias usadas solo en análisis/entrenamiento/experimentación (notebooks, matplotlib, MLflow completo, etc.). La imagen bajó de 1.37 GB a 707 MB manteniendo el mismo comportamiento.
- `.dockerignore` independiente de `.gitignore`: evita incluir en el contexto de build datasets, notebooks, artefactos de MLflow y entornos virtuales, aunque esos archivos sí puedan estar versionados o presentes localmente, ambos mecanismos resuelven problemas distintos (qué se sube a Git vs. qué entra al build de Docker).
- Excepción puntual en `.gitignore` para versionar el modelo final (bug encontrado y corregido): la regla general `*.joblib` impedía que `models/random_forest_model.joblib` se subiera a GitHub, aunque `.dockerignore` ya tenía una excepción para el build local. Se detectó al clonar el repositorio en limpio y comprobar que `docker build` fallaría para cualquier persona ajena al entorno local: no existía el archivo del modelo. Se corrigió agregando `!models/random_forest_model.joblib` y versionando el archivo (25 MB), y se re-verificó con un clon limpio adicional más un ciclo completo de build/run/predict.

12. Monitoring

El sistema incorpora monitoreo en producción en tres niveles, métricas operativas, drift de datos y desempeño del modelo, más monitoreo de calidad sobre los batches de producción.

12.1 System Monitoring

Implementado en `src/monitoring/system_monitor.py`. La API registra automáticamente latency, throughput, error rate y availability, consultables vía `GET /metrics`.

Decisión tomada

- Excluir `/metrics` del propio conteo: evita que la consulta de observabilidad distorsione las métricas que está midiendo (el acto de medir no debe afectar lo medido).

12.2 Reference y Production Batches

Implementado en `src/monitoring/production_batches.py`. División estrictamente temporal: el 85 % histórico (usado en train+validation) es REFERENCE; el 15 %

final, nunca usado para entrenar, se divide cronológicamente en tres batches de producción.

Conjunto	Filas
REFERENCE	28,879
PRODUCTION_BATCH_1	1,699
PRODUCTION_BATCH_2	1,699
PRODUCTION_BATCH_3	1,699

12.3 Data Drift

Implementado en `src/monitoring/drift_monitor.py` mediante Population Stability Index (PSI), comparando cada batch de producción contra REFERENCE.

Rango de PSI	Veredicto
$PSI < 0.10$	OK
$0.10 \leq PSI < 0.25$	WARNING
$PSI \geq 0.25$	ALERT

Además se incluye una simulación controlada de drift (incremento del 35 % sobre variables históricas de consumo en una copia de PRODUCTION_BATCH_3), sin alterar el dataset original.

Decisiones tomadas y justificación

- Umbrales de PSI como criterio operativo, no como límite universal: una alerta de drift señala un cambio de distribución, pero no implica por sí sola que el modelo haya dejado de funcionar, de ahí la separación con Model Monitoring y el Retraining Trigger.
- Simular drift sobre una copia, nunca sobre el dataset original: permite demostrar que el sistema de detección funciona ante un cambio real y significativo, sin comprometer la integridad de los datos de referencia.

12.4 Model Monitoring

Implementado en `src/monitoring/model_monitor.py`. Cuando el ground truth está disponible, se calculan MAE_t y RMSE_t por batch:

Batch	MAE_t (kW)	RMSE_t (kW)
PRODUCTION_BATCH_1	0.4317	0.5820
PRODUCTION_BATCH_2	0.3968	0.5256

PRODUCTION_BATCH_3	0.4708	0.6571
--------------------	--------	--------

Decisión tomada

- Medir desempeño real por batch, no solo distribución de entradas: permite distinguir explícitamente entre un cambio en los datos (drift) y una degradación real del desempeño predictivo, son fenómenos distintos y el proyecto los trata como tales.

12.5 Production Data Quality Monitoring

Implementado en `src/monitoring/quality_monitor.py`. Ante una copia contaminada de PRODUCTION_BATCH_3 (valor faltante, fila duplicada, outlier extremo, datatype incorrecto, columna inesperada), el monitor aplica el flujo Detect → Block / Warn → Log, registrando evidencia en `reports/monitoring/quality_report.json`.

Decisión tomada

- BLOCK para problemas incompatibles con inferencia segura (schema, missing, tipos); WARN para duplicados/outliers: un dato con tipo incorrecto o schema roto no puede procesarse de forma confiable y debe bloquearse; un duplicado o un outlier extremo amerita revisión pero no necesariamente invalida la entrada.

12.6 Retraining Trigger

Implementado en `src/monitoring/retraining_trigger.py`. La decisión de reentrenar combina dos señales, nunca una sola:

significant_drift AND performance_degradation → RETRAIN

La degradación se calcula comparando MAE_t/RMSE_t contra el desempeño de referencia en test, con hasta 20 % de tolerancia. Ejemplo real con PRODUCTION_BATCH_3 (drift significativo detectado, pero desempeño estable):

Drift significativo: True

MAE actual: 0.4708 (threshold: 0.5197)

RETRAIN: False

Razón: existe drift significativo, pero el desempeño del modelo permanece dentro de los límites aceptables. No se reentrena.

Decisión tomada y justificación

- Drift por sí solo nunca dispara el reentrenamiento: un cambio en la distribución de los datos no implica necesariamente que el modelo haya dejado de predecir correctamente; reentrenar automáticamente ante cualquier drift generaría reentrenamientos innecesarios y costosos. El caso

real documentado (PRODUCTION_BATCH_3) demuestra exactamente ese escenario: drift detectado, pero desempeño dentro de tolerancia → no se reentrena.

13. Interfaz y despliegue (complemento del proyecto)

Se construyó una interfaz visual y su despliegue público, para facilitar la demostración del proyecto.

13.1 Interfaz Streamlit

Implementada en ui/app.py. Consume la API únicamente vía HTTP (GET /health, GET /model-info, POST /predict), no importa el modelo directamente. Muestra el estado de la API/modelo y un formulario para generar un pronóstico a 24 horas sin construir el JSON manualmente.

Decisión tomada

- La UI nunca importa el modelo directamente, solo consume la API por HTTP: mantiene una separación estricta de responsabilidades: la UI es un cliente cualquiera de la API, igual que lo sería un sistema externo, si el modelo cambia de formato o de ubicación, la UI no se ve afectada.

13.2 Despliegue en Render

El archivo render.yaml (Blueprint de Render) define ambos servicios, la API (Docker, reutilizando el Dockerfile ya validado) y la interfaz Streamlit, desplegables con un solo Blueprint.

Servicio	URL pública
household-power-api	https://household-power-api-6w5t.onrender.com
household-power-ui	https://household-power-ui.onrender.com

El despliegue fue verificado end-to-end: /health responde 200 con model_loaded=true, y la UI genera un pronóstico real consumiendo la API pública desplegada.

Decisiones tomadas y justificación

- API_URL como variable sync: false en el Blueprint: su valor depende de la URL pública que Render asigna a la API, la cual no se conoce hasta después del primer deploy, se completa manualmente una sola vez desde el dashboard tras el primer despliegue de la API.
- Aceptar las limitaciones del plan gratuito (por inactividad): es una demo académica, no un servicio productivo; se documenta explícitamente que la primera solicitud tras inactividad puede tardar 30-60 segundos, y se recomienda abrir el link minutos antes de la demo.

14. Estrategia de ramas y colaboración

El proyecto siguió un flujo de Git de tres niveles:

main ← develop ← feature/fix/docs/chore/...

- **main:** versión estable, solo recibe cambios mediante Pull Request desde develop.
- **develop:** rama de integración; recibe los Pull Requests de cada rama de trabajo.
- **feature/..., fix/..., docs/..., chore/...:** una rama por tarea concreta, revisada en Pull Request antes de fusionarse a develop.

Ambos integrantes participamos a lo largo de todas las etapas del proyecto (ingesta, calidad de datos, EDA, feature engineering, modelado, MLflow, API, Docker, monitoreo y despliegue), trabajando en ramas independientes y revisando el trabajo del otro antes de fusionar, incluyendo la corrección mutua de errores encontrados en el trabajo del compañero (por ejemplo, el fix de leakage en el target de Feature Engineering, sección 7, y el fix de archive_existing_versions en el Model Registry, sección 9).

Decisión tomada y justificación

- Nunca hacer push directo a main o develop: todo cambio pasa por Pull Request, incluso entre los dos únicos integrantes del equipo, esto nos permite revisión cruzada antes de integrar, y deja un historial auditable de qué cambió, por qué y quién lo aprobó.
- Sincronizar develop: main mediante PRs de release periódicos, no en cada commit: evita abrir un Pull Request a main por cada cambio menor (p. ej. una corrección de README); se consolidan varios cambios de develop en un solo PR de sincronización antes de un hito importante (como la entrega o la demo).

15. Verificación de reproducibilidad

A lo largo del proyecto se aplicó de forma sistemática un principio de 'verificar antes de afirmar': cada componente crítico se probó en un entorno limpio antes de darlo por funcional.

- Clonado limpio del repositorio (sin ningún archivo local previo) + entorno virtual nuevo + ejecución completa del pipeline de principio a fin, obteniendo resultados idénticos a los reportados en este informe.
- Verificación explícita de que docker build funciona para cualquier persona que clone el repositorio por primera vez, sin depender de artefactos generados localmente que no estuvieran versionados (ver corrección del bug crítico en la sección 11).

- Reentrenamiento repetido (3 veces) para confirmar que el Model Registry mantiene un único modelo en estado Production tras la corrección del bug de `archive_existing_versions`.
- Verificación end-to-end del despliegue en Render: API respondiendo `/health` con el modelo cargado, y la UI generando un pronóstico real consumiendo la API pública.

16. Conclusiones

- El proyecto cubre el ciclo completo de MLOps solicitado: ingesta, validación, limpieza, EDA, feature engineering, entrenamiento, tracking y registro de modelos, empaquetado, servicio de inferencia, pruebas automatizadas, monitoreo (drift, desempeño, calidad, sistema) y una estrategia explícita de reentrenamiento.
- Cada decisión de diseño relevante, desde los umbrales de validación hasta el criterio de reentrenamiento, está justificada con evidencia extraída del propio dataset, no adoptada por defecto.
- El proceso de desarrollo colaborativo incluyó revisión cruzada del trabajo entre ambos integrantes, lo que permitió detectar y corregir errores no triviales (leakage de target, versiones múltiples de Production en el Model Registry, y un bug crítico de `.gitignore` que habría roto el build de Docker para cualquier persona externa al entorno local).
- El sistema final es reproducible de extremo a extremo: fue verificado mediante clones limpios del repositorio, entornos virtuales nuevos y un despliegue público funcional en Render.

17. Equipo

Proyecto desarrollado por el Grupo 7.

Integrante	GitHub
Breidy Bonilla	@BreidyBR
Oscar Eduardo Sánchez Barahona	@oscar2903

Ambos integrantes participaron a lo largo de todas las etapas del proyecto, trabajando mediante ramas independientes y Pull Requests revisados antes de fusionarse a `develop`, y de `develop` a `main` una vez verificado el proyecto completo.